

Support de Cours RAG avec Python

Retrieval-Augmented Generation - du LLM a la base vectorielle

Niveau : Debutant

Outils : Groq, FAISS, sentence-transformers

Langage : Python 3.10+

Support de Cours : Comprendre et Construire un RAG

De l'API LLM a un systeme de Retrieval-Augmented Generation complet

Sommaire

1. Vue d'ensemble : Qu'est-ce qu'un RAG ?
 2. Utiliser un LLM via une API (Groq)
 3. Les Embeddings - Transformer du texte en vecteurs
 4. Les Bases de Donnees Vectorielles
 5. Le Chunking - Decouper les documents intelligemment
 6. Pipeline RAG complet - Tout assembler
 7. Ameliorations avancees
 8. Recapitulatif et bonnes pratiques
-

1. Vue d'ensemble : Qu'est-ce qu'un RAG ?

Le probleme de base

Un LLM (Large Language Model) comme GPT-4 ou Llama est entraine sur des donnees jusqu'a une certaine date. Il ne connait donc pas vos documents internes, les informations recentes posterieures a son entraînement, ni les donnees specifiques a votre domaine. Si vous lui posez une question sur votre documentation interne, il va soit **inventer une reponse** (c'est ce qu'on appelle une hallucination), soit admettre qu'il ne sait pas.

La solution : le RAG

Le **RAG (Retrieval-Augmented Generation)** est une technique qui permet d'enrichir le contexte d'un LLM avec des informations recuperees dynamiquement depuis une base de connaissances. En pratique : on cherche les bons documents dans la base, puis on les donne au LLM pour qu'il formule une reponse fiable.

Analogie : imaginez un etudiant qui repond a une question d'examen. Sans RAG, il repond de memoire et peut se tromper. Avec RAG, il a le droit de consulter les bons chapitres du cours avant de repondre.

Architecture globale : deux phases distinctes

Un systeme RAG fonctionne en deux grandes phases bien separees :

Phase 1 - Indexation (effectuee une seule fois au depart)

- Les documents sources sont charges et nettoyes
- Chaque document est decoupe en petits morceaux appeles **chunks**
- Chaque chunk est converti en vecteur numerique par un modele d'**embedding**
- Ces vecteurs sont stockes dans une **base vectorielle** indexee

Phase 2 - Interrogation (executee a chaque nouvelle question)

- La question de l'utilisateur est convertie en vecteur (avec le meme modele)
- La base vectorielle recherche les chunks semantiquement proches
- Les meilleurs chunks sont selectionnes et passes au LLM comme contexte
- Le LLM genere une reponse fondee sur ce contexte, et cite ses sources

2. Utiliser un LLM via une API (Groq)

Pourquoi Groq ?

Groq est une plateforme qui propose un acces API a des modeles open-source tels que Llama 3 et Mixtral. Elle se distingue par une vitesse d'inference particulierement elevee et par un **tier gratuit genereux**, ce qui en fait le choix ideal pour debuter sans frais.

Les parametres cles d'un appel API

Parametre	Role	Valeur conseilee
model	Quel LLM utiliser	llama3-8b-8192 (rapide) ou mixtral-8x7b-32768
temperature	Niveau de creativite du modele	0.0 pour du factuel, 0.7 pour du conversationnel
max_tokens	Longueur maximale de la reponse	512 a 2048 selon le besoin
messages	Historique de la conversation	Liste de dictionnaires {role, content}

Les trois roles dans les messages

Chaque appel a l'API se compose d'une liste de messages. Chaque message possede un **role** qui determine son interpretation :

- **system** : definit le comportement general du LLM (instructions permanentes, ton, contraintes)
- **user** : la question ou le message de l'utilisateur
- **assistant** : la reponse precedente du LLM (pour gerer l'historique)

Point important : le LLM n'a aucune memoire entre deux appels distincts. Pour simuler une conversation coherente, vous devez envoyer l'integralite de l'historique a chaque nouvel appel.

Injecter du contexte dans le prompt (base du RAG)

La mecanique fondamentale d'un RAG consiste a inserer les chunks retrouves dans le prompt envoye au LLM. Le prompt systeme doit explicitement instruire le modele de **repondre uniquement en se basant sur le contexte fourni**, de citer ses sources, et de refuser d'inventer si l'information est absente du contexte.

3. Les Embeddings - Transformer du texte en vecteurs

L'idée fondamentale

Un **embedding** est une représentation numérique du sens d'un texte. On transforme une phrase en une liste de nombres (appelé vecteur) de telle sorte que deux phrases au sens proche produisent des vecteurs proches dans l'espace mathématique.

Exemple concret : les phrases "Le chat dort sur le canapé" et "Un félin sommeille sur le sofa" n'ont presque aucun mot en commun, mais leurs vecteurs seront très proches car elles expriment la même idée. À l'inverse, "La voiture roule vite" produira un vecteur très éloigné.

Analogie : imaginez placer chaque phrase sur une carte géographique. Deux phrases au sens proche se retrouvent dans la même ville. Une phrase sur un sujet différent se trouve dans un autre pays.

Pourquoi c'est utile pour le RAG ?

Quand un utilisateur pose une question, on peut retrouver les documents les plus pertinents **sans correspondance exacte de mots**. Par exemple, la question "Quel est le délai de remboursement dentaire ?" peut retrouver un chunk contenant "Les soins bucco-dentaires sont pris en charge sous 30 jours" - sans avoir un seul mot en commun.

La similarité cosinus

C'est la mesure mathématique utilisée pour comparer deux vecteurs. Elle calcule l'angle entre eux. Un score de 1.0 signifie des textes identiques, un score proche de 0.9 indique des textes très proches sur le même sujet, un score autour de 0.5 signale des textes liés mais différents, et un score proche de 0 indique des textes sans rapport.

La règle fondamentale : même modèle pour tout

On doit impérativement utiliser **le même modèle d'embedding** pour vectoriser les documents lors de l'indexation ET pour vectoriser la question lors de l'interrogation. C'est ce qui garantit la cohérence de l'espace vectoriel et permet de comparer les vecteurs de manière significative.

La dilution sémantique : un piège avec les questions longues

Un vecteur représente **un seul point** dans l'espace mathématique. Si la question est très longue et contient plusieurs idées, le vecteur devient une moyenne floue de toutes ces idées, ce qui dégrade la qualité de la recherche. Une question courte et précise produit un vecteur cible ; une question longue et complexe produit un vecteur imprécis. La solution est de décomposer les questions longues en sous-questions atomiques (voir section 7).

Choisir son modele d'embedding

Modele	Langue	Dimensions	Limite tokens	Usage
all-MiniLM-L6-v2	Anglais	384	512	Prototypage rapide
all-mpnet-base-v2	Anglais	768	514	Meilleure qualite EN
paraphrase-multilingual-mpnet-base-v2	Multilingue	768	514	Francais recommande
text-embedding-3-small (OpenAI)	Multilingue	1536	8191	Production, payant
nomic-embed-text	Multilingue	768	8192	Local, excellent

Pour du texte en francais, privilegiez toujours un modele multilingue.

4. Les Bases de Donnees Vectorielles

Pourquoi pas une base SQL classique ?

Une base SQL cherche par correspondance exacte de mots. Elle ne retrouve que les documents contenant precisement les termes de la requete - pas les synonymes, pas les reformulations. Une base vectorielle, elle, cherche par **proximite semantique** : elle compare des vecteurs et retourne les plus proches, independamment des mots exacts utilises.

Comment ca fonctionne en interne ?

Comparer la question a chaque vecteur stocke serait beaucoup trop lent sur des millions de documents. Les bases vectorielles utilisent des **algorithmes d'indexation approximatifs (ANN)** comme HNSW (Hierarchical Navigable Small World) pour naviguer efficacement dans l'espace des vecteurs. Le resultat : une recherche qui prendrait des minutes en force brute se fait en quelques millisecondes.

FAISS - La solution locale pour debuter

FAISS (developpee par Meta) est une bibliotheque performante qui fonctionne directement en memoire. Elle est gratuite, open-source, et parfaite pour developper et tester un RAG localement. Elle permet de creer un index, d'y ajouter des vecteurs, de sauvegarder l'index sur disque et de le recharger sans avoir a reindexer.

L'importance des metadonnees

Il est essentiel de stocker des metadonnees avec chaque chunk : la source du document, le numero de page, la section, la date d'ajout... Ces informations permettent au LLM de **citer ses sources** dans la reponse finale, et a l'utilisateur de verifier l'information.

Comparaison des solutions

Solution	Type	Avantages	Inconvenients	Usage ideal
FAISS	Locale	Gratuit, rapide, simple	Pas de persistance native	Prototypage
ChromaDB	Locale	Simple, persistance facile	Moins scalable	Projets moyens
Pinecone	Cloud	Scalable, entierement gere	Payant	Production large echelle
Weaviate	Cloud/Local	Open-source, riche	Complexe a configurer	Production avantee
Qdrant	Cloud/Local	Rapide, filtres puissants	Moins connu	Production moderne

5. Le Chunking - Decouper les documents intelligemment

Pourquoi decouper ?

Les modeles d'embedding ont une limite de tokens : on ne peut pas vectoriser un document entier de 100 pages en un seul vecteur. De plus, un vecteur tres long serait trop vague pour etre utile a la recherche. L'objectif du chunking est de creer des morceaux de texte suffisamment petits pour etre embeddees avec precision, mais suffisamment grands pour contenir une idee complete.

Strategie 1 : Chunking naif par taille fixe

DECONSEILLE. Le texte est decoupe tous les N caracteres sans tenir compte du sens. Une phrase peut etre coupee en plein milieu, rendant les chunks incoherents et nuisant a la qualite de la recherche.

Strategie 2 : RecursiveCharacterTextSplitter

Le texte est decoupe en respectant une hierarchie de separateurs naturels dans l'ordre suivant : paragraphes, puis lignes, puis phrases, puis mots. Un **chevauchement (overlap)** est applique : les derniers N caracteres du chunk precedent sont repetes au debut du suivant, evitant ainsi la perte d'information aux jointures.

Strategie 3 : Chunking par structure du document

La hierarchie naturelle du document (titres H1, H2, H3, sections) est utilisee comme guide de decoupe. Chaque chunk conserve ses metadonnees de section (titre, sous-titre). Ideal pour les documents Markdown, HTML ou les articles juridiques.

Strategie 4 : Chunking semantique

Le texte est decoupe **uniquement quand le sens change**. On compare la similarite cosinus entre phrases consecutives : si la similarite chute fortement, on insere un point de coupure. Limite : un embedding mesure une proximite de surface et ne comprend pas vraiment les structures argumentatives complexes (these/antithese/synthese).

Strategie 5 : Parent-Child Chunking (recommande en production)

Deux niveaux de granularite sont utilises. Les **petits chunks** (200 caracteres) sont indexes dans la base vectorielle pour une recherche precise. Quand un petit chunk est retrouve, c'est son **grand parent** (2000 caracteres) qui est envoye au LLM pour lui donner plus de contexte. C'est le meilleur compromis entre precision de recherche et richesse du contexte fourni.

Comparaison des strategies

Strategie	Qualite	Cout	Usage ideal
Taille fixe	Mauvaise	Gratuit	A éviter
Recursive + overlap	Bonne	Gratuit	Prototypage
Par structure	Tres bonne	Gratuit	Docs structures
Semantique	Excellente	Faible	Texte brut homogene
Parent-Child	Optimale	Faible	Production
LLM chunking	Maximale	Eleve	Documents critiques

Gerer des documents de formats differents

En production, vous aurez des PDF, des fichiers Word, du Markdown, du HTML... La bonne pratique est de mettre en place un pipeline de **detection automatique du format** : selon l'extension ou la structure du document, le bon chargeur (loader) et la bonne strategie de chunking sont selectionnees automatiquement.

6. Pipeline RAG complet - Tout assembler

Un pipeline RAG complet assemble toutes les briques precedentes en deux scripts distincts :

Script d'indexation (execute une seule fois)

- Charger les documents depuis le disque (PDF, Word, Markdown...)
- Nettoyer et normaliser le texte
- Decouper en chunks avec la strategie choisie
- Embedder chaque chunk avec le modele sentence-transformers
- Creer l'index FAISS et le sauvegarder sur disque avec les metadonnees

Script de questions-reponses (execute a chaque session)

- Charger l'index FAISS depuis le disque (pas besoin de reindexer)
- Lire la question de l'utilisateur
- Embedder la question avec le meme modele
- Rechercher les k chunks les plus proches (ex: k=20)
- Reranker pour ne garder que les 4 vraiment pertinents
- Construire le prompt avec le contexte et les sources
- Appeler l'API Groq et retourner la reponse avec les citations

Points clés de conception

Le prompt systeme est critique : il doit explicitement interdire au LLM d'inventer des informations, lui imposer de citer le document et la page source pour chaque affirmation, et lui demander de dire clairement "je ne trouve pas cette information" si le contexte est insuffisant. La temperature doit etre basse (0.0 a 0.2) pour privilegier la fidelite au contexte sur la creativite.

7. Ameliorations avancees

7.1 Le Reranking - Affiner la selection des chunks

La recherche vectorielle par similarite cosinus est rapide mais approximative. Elle peut remonter des chunks thematiquement proches mais pas vraiment pertinents pour la question posee.

Le **reranker** (ou cross-encoder) est un second modele qui analyse la paire (question, chunk) **ensemble** et attribue un score de pertinence bien plus fiable. Contrairement a l'embedding qui encode les deux textes separement, le reranker les lit simultanément et comprend leur relation.

La strategie optimale en production est de combiner les deux : la recherche vectorielle fait un **premier filtre rapide** sur les 20 meilleurs candidats, puis le reranker effectue une **selection fine** pour ne garder que les 4 vraiment pertinents. Le gain de qualite typique est de 20 a 40% sur les benchmarks standard.

Approche	Vitesse	Precision	Principe
Bi-encoder (embedding)	Tres rapide (50ms)	Approximative	Question et chunk encodes separement, comparaison cosinus
Cross-encoder (reranker)	Lent (200ms)	Tres precise	Question et chunk lus ensemble par le modele
Combine (recommande)	Rapide (250ms total)	Optimale	Embedding pour filtrer, reranker pour classer

Pour du francais, l'API Cohere Rerank (rerank-multilingual-v3.0) est particulierement efficace et facile a integrer.

7.2 HyDE - Hypothetical Document Embedding

Le probleme : le vecteur d'une question et le vecteur d'une reponse ne se ressemblent pas forcément, meme s'ils parlent du meme sujet. Les chunks indexes ressemblent a des reponses, pas a des questions.

La solution HyDE : au lieu d'embedder la question directement, on demande d'abord au LLM de **generer une reponse hypothetique** (meme approximative), puis on embedde cette reponse. Comme elle ressemble structurellement aux chunks indexes, la recherche est plus pertinente.

7.3 Decomposition des questions longues

Comme vu en section 3, une question longue produit un vecteur "dilue". La solution : on demande au LLM de **decomposer la question en 2 a 4 sous-questions simples et independantes**. Chaque sous-question est embeddee separement, ce qui donne des vecteurs precis. Les resultats de toutes les recherches sont agrages et dedupliques avant d'etre passes au LLM.

8. Recapitulatif et bonnes pratiques

Le pipeline RAG production complet

Voici le pipeline recommande pour un systeme RAG pret pour la production :

Indexation (une seule fois) :

- Documents sources -> Detection du format -> Loader adapte
- Chunking Parent-Child ou Semantique selon le type de contenu
- Embedding avec un modele multilingue
- Stockage dans la base vectorielle avec metadonnees completes

Interrogation (a chaque question) :

- Si question longue : decomposition en sous-questions
- Embedding de la (des) question(s)
- Recherche vectorielle -> top 20 candidats
- Reranking -> top 4 pertinents
- Construction du prompt avec contexte et sources
- LLM -> reponse citant ses sources

Bonnes pratiques

A faire	A eviter
Stocker les metadonnees (source, page, section)	Chunking naif par taille fixe
Meme modele d'embedding pour docs et questions	Changer de modele apres indexation
Implementer un reranker en production	Se fier uniquement a la similarite cosinus
Tester la qualite des chunks manuellement	Ignorer la qualite du chunking
Demander au LLM de citer ses sources	Laisser le LLM improviser hors contexte
Prevoir un fallback si aucun chunk pertinent	Envoyer un contexte vide au LLM
Monitorer les requetes et les reponses	Deployer sans systeme d'evaluation

Checklist pour votre premier RAG

- Cle API Groq obtenue (gratuit sur console.groq.com)
- Documents sources prepares et nettoyes
- Strategie de chunking choisie selon le type de documents
- Modele d'embedding adapte a la langue (multilingue pour le francais)
- Base vectorielle creee et testee sur quelques requetes manuelles
- Prompt systeme clair : ne pas halluciner, citer les sources
- Reranker ajoute si mise en production
- Tests manuels avec des questions variees avant deploiement

Ressources pour aller plus loin

- **LangChain** (python.langchain.com) : framework RAG complet
- **LlamaIndex** (docs.llamaindex.ai) : alternative puissante
- **Groq** (console.groq.com) : API LLM rapide et gratuite
- **MTEB Leaderboard** (huggingface.co/spaces/mteb/leaderboard) : classement des modeles d'embedding
- **Ragas** (docs.ragas.io) : framework d'evaluation de RAG